SQL DATA ANALYSIS PROJECT

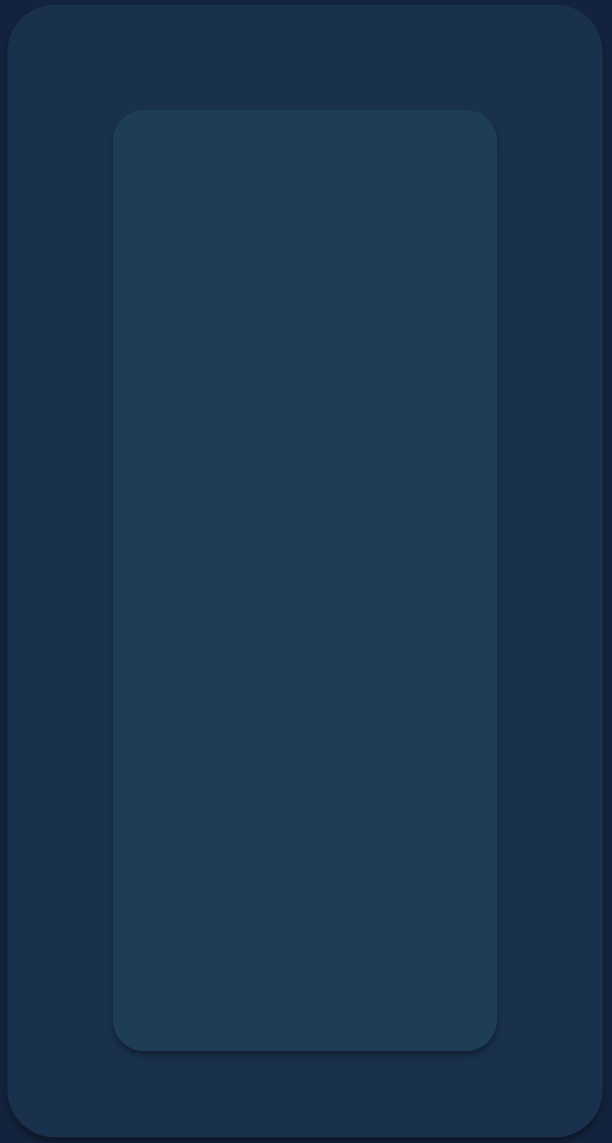
ONLINE BOOK STORE

Basic & Advanced SQL Queries

Presented by

Sanjay Kumar Thakur

PostgreSQL • Query Analysis • Results



QUERY WALKTHROUGH

Project Overview

This project analyzes an Online Book Store database using PostgreSQL to explore book inventory, sales, revenue, and customer purchasing behavior. I used basic and advanced SQL queries to answer business questions and extract meaningful insights from the data.

PROJECT OBJECTIVE

- Analyze book inventory, genres, and pricing.
- Evaluate sales performance and revenue.
- Identify popular books and genres.
- Understand customer purchasing behavior.
- Analyze remaining stock after fulfilling orders.

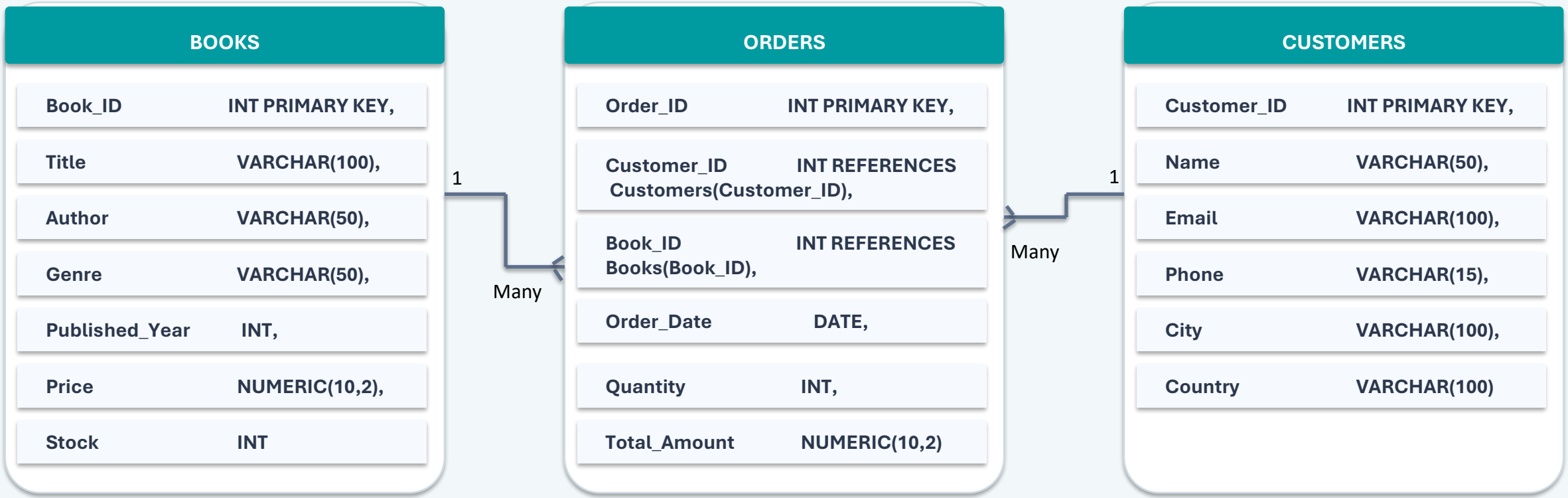
TOOLS & TECHNOLOGY

- PostgreSQL
- SQL
- pgAdmin

DATABASE TABLES

- Books
- Customers
- Orders

Database Structure



Relationships: Books.book_id → Orders.book_id | Customers.customer_id → Orders.customer_id

SECTION 01

BASIC SQL QUERIES

1) Retrieve all books in the 'Fiction' genre
2) Find books published after the year 1950
3) List all customers from the Canada
4) Show orders placed in November 2023
5) Retrieve the total stock of books available
6) Find the details of the most expensive books
7) Show all customers who ordered more than 1 quantity of a book
8) Retrieve all orders where the total amount exceeds $20
9) List all genres available in the Books table
10) Find the book with the lowest stock
11) Calculate the total revenue generated from all orders

Basic Query 01) Retrieve all books in the 'Fiction' genre

SQL QUERY

```
SELECT *
FROM Books
WHERE genre='Fiction';
```

QUERY RESULT

	book_id [PK] integer	title character varying (100)	author character varying (50)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.52	8
2	22	Multi-layered optimizing migration	Wesley Escobar	Fiction	1908	39.23	78
3	28	Expanded analyzing portal	Lisa Coffey	Fiction	1941	37.51	79
4	29	Quality-focused multi-tasking challenge	Katrina Underwood	Fiction	1905	31.12	100
5	31	Implemented encompassing conglomerat...	Melissa Taylor	Fiction	2010	21.23	44
6	39	Optimized national process improvement	Megan Goodwin	Fiction	1978	10.99	42

Total rows: 60 Query complete 00:00:00.347

Basic Query 02) Find books published after the year 1950

SQL QUERY

```
SELECT book_id, title, author, published_year
FROM Books
WHERE published_year>1950;
```

QUERY RESULT

	book_id [PK] integer	title character varying (100)	author character varying (50)	published_year integer
1	2	Persevering reciprocal knowledge user	Mario Moore	1971
2	4	Customizable 24hour product	Christopher Andrews	2020
3	5	Adaptive 5thgeneration encoding	Juan Miller	1956
4	6	Advanced encompassing implementation	Bryan Morgan	1985
5	8	Persistent local encoding	Troy Cox	2019
6	9	Optimized interactive challenge	Colin Buckley	1987

Total rows: 292 Query complete 00:00:00.154

Basic Query 03) List all customers from the Canada

SQL QUERY

```
SELECT customer_id, name, country
FROM customers
WHERE country='Canada';
```

QUERY RESULT

	customer_id [PK] integer	name character varying (50)	country character varying (100)
1	38	Nicholas Harris	Canada
2	415	James Ramirez	Canada
3	468	David Hart	Canada

Basic Query 04) Show orders placed in November 2023

SQL QUERY

```
SELECT * FROM Orders
WHERE order_date BETWEEN '2023-11-01' AND '2023-11-30';
```

QUERY RESULT

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	4	433	343	2023-11-25	7	301.21
2	19	496	60	2023-11-17	9	316.26
3	75	291	375	2023-11-30	5	170.75
4	132	469	333	2023-11-22	7	194.32
5	137	474	471	2023-11-25	8	363.04
6	163	207	384	2023-11-23	3	101.76

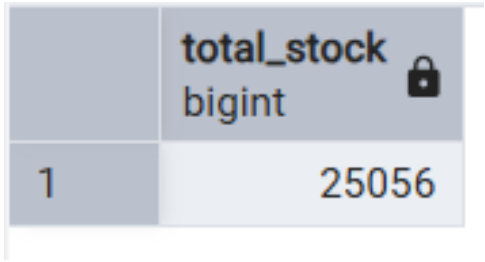
Total rows: 25 Query complete 00:00:00.128

Basic Query 05) Retrieve the total stock of books available

SQL QUERY

```
SELECT SUM(stock) AS Total_Stock
FROM Books;
```

QUERY RESULT



	total_stock bigint
1	25056

Basic Query 06) Find the details of the most expensive books

SQL QUERY

```
SELECT * FROM Books
ORDER BY price DESC
LIMIT 1;
```

QUERY RESULT

	book_id [PK] integer	title character varying (100)	author character varying (50)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	340	Proactive system-worthy orchestrati...	Robert Scott	Mystery	1907	49.98	88

Basic Query 07) Show all customers who ordered more than 1 quantity of a book

SQL QUERY

```
SELECT * FROM orders
WHERE quantity>1;
```

QUERY RESULT

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	1	84	169	2023-05-26	8	188.56
2	2	137	301	2023-01-23	10	216.60
3	3	216	261	2024-05-27	6	85.50
4	4	433	343	2023-11-25	7	301.21
5	5	14	431	2023-07-26	7	136.36
6	6	439	119	2024-10-11	5	249.40

Total rows: 438 Query complete 00:00:00.196

Basic Query 08) Retrieve all orders where the total amount exceeds $20

SQL QUERY

```
SELECT * FROM Orders
WHERE total_amount>20;
```

QUERY RESULT

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	1	84	169	2023-05-26	8	188.56
2	2	137	301	2023-01-23	10	216.60
3	3	216	261	2024-05-27	6	85.50
4	4	433	343	2023-11-25	7	301.21
5	5	14	431	2023-07-26	7	136.36
6	6	439	119	2024-10-11	5	249.40

Total rows: 473 Query complete 00:00:00.316

Basic Query 09) List all genres available in the Books table

SQL QUERY

```
SELECT DISTINCT genre
FROM Books;
```

QUERY RESULT



	genre character varying (50)
1	Romance
2	Biography
3	Mystery
4	Fantasy
5	Fiction
6	Non-Fiction

Total rows: 7 Query comp

Basic Query 10) Find the book with the lowest stock

SQL QUERY

```
SELECT * FROM Books
ORDER BY stock
LIMIT 11;
```

QUERY RESULT

	book_id [PK] integer	title character varying (100)	author character varying (50)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	60	Robust eco-centric capacity	Brian Haney	Biography	1990	35.14	0
2	378	Future-proofed heuristic function	Samantha Mcclain	Romance	1903	6.01	0
3	163	Object-based eco-centric challenge	Douglas Mccarthy	Non-Fiction	1905	19.11	0
4	127	Business-focused real-time benchmark	David Nelson	Science Fiction	1997	11.66	0
5	44	Networked systemic implementation	Ryan Frank	Science Fiction	1965	13.55	0
6	19	Progressive asymmetric Internet solut...	Sean Miller	Science Fiction	1990	11.31	1

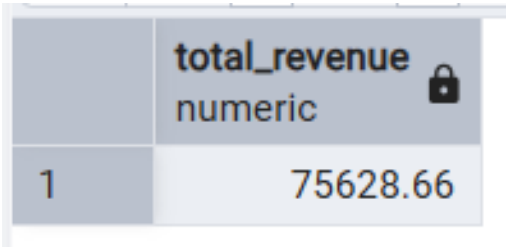
Total rows: 11 Query complete 00:00:00.205

Basic Query 11) Calculate the total revenue generated from all orders

SQL QUERY

```
SELECT SUM(total_amount) AS Total_Revenue
FROM Orders;
```

QUERY RESULT



	total_revenue numeric
1	75628.66

SECTION 02

ADVANCED SQL QUERIES

1) Retrieve the total number of books sold for each genre

2) Find the average price of books in the 'Fantasy' genre

3)List customers who have placed at least 2 orders

4) Find the most frequently Ordered book

5) Show the top 3 most expensive books of 'Fantasy' genre

6) Retrieve the total quantity of books sold by each author

7) List the cities where customers who spent over $30 are located

8) Find the customer who spent the most on orders

9) Calculate stock remaining after fulfilling all orders

Advanced Query 01) Retrieve the total number of books sold for each genre

SQL QUERY

```
SELECT B.genre, SUM(O.quantity) AS Total_Book_Sold
FROM Books AS B
JOIN Orders AS O
ON B.book_id=O.book_id
GROUP BY B.genre;
```

QUERY RESULT

	genre character varying (50)	total_book_sold bigint
1	Romance	439
2	Biography	285
3	Mystery	504
4	Fantasy	446
5	Fiction	225
6	Non-Fiction	351

Total rows: 7 Query complete 00:00:00.158

Advanced Query 02) Find the average price of books in the 'Fantasy' genre

SQL QUERY

```
SELECT genre, AVG(price) AS Average_Price
FROM Books
WHERE genre='Fantasy'
GROUP BY genre;
```

QUERY RESULT

	genre character varying (50)	average_price numeric
1	Fantasy	25.981690140845070400

Advanced Query 03) List customers who have placed at least 2 orders

SQL QUERY

```
SELECT C.customer_id, C.name, COUNT(O.order_id) AS Order_Count
FROM Customers AS C
JOIN Orders AS O
ON C.customer_id=O.customer_id
GROUP BY C.customer_id, C.name
HAVING COUNT(O.order_id)>=2
ORDER BY COUNT(O.order_id) DESC;
```

QUERY RESULT

	customer_id [PK] integer	name character varying (50)	order_count bigint
1	364	Carrie Perez	6
2	474	Anthony Young	5
3	405	Julie Smith	4
4	485	Andrew Figueroa	4
5	457	Kim Turner	4
6	107	Amy Hunt	4

Total rows: 139 Query complete 00:00:00.199

Advanced Query 04) Find the most frequently Ordered book

SQL QUERY

```
SELECT O.book_id, B.title, B.genre, COUNT(O.order_id) AS Order_Count
FROM Books AS B
JOIN Orders AS O
ON B.book_id=O.book_id
GROUP BY O.book_id, B.title,B.genre
ORDER BY Order_Count DESC;
```

QUERY RESULT

	book_id integer	title character varying (100)	genre character varying (50)	order_count bigint
1	88	Robust tangible hardware	Non-Fiction	4
2	120	Integrated secondary access	Non-Fiction	4
3	273	Devolved zero administration process improvement	Romance	4
4	31	Implemented encompassing conglomeration	Fiction	4
5	491	Pre-emptive intangible adapter	Fantasy	4
6	333	Advanced responsive extranet	Fantasy	4
7	73	Realigned multi-tasking installation	Mystery	4
8	208	Synergized intangible methodology	Fantasy	3

Advanced Query 05) Show the top 3 most expensive books of 'Fantasy' genre

SQL QUERY

```
SELECT book_id, title, price
FROM Books
WHERE genre='Fantasy'
ORDER BY price DESC
LIMIT 3;
```

QUERY RESULT

	book_id [PK] integer	title character varying (100)	price numeric (10,2)
1	240	Stand-alone content-based hub	49.90
2	462	Innovative 3rdgeneration databa...	49.23
3	238	Optimized even-keeled analyzer	48.97

Advanced Query 06) Retrieve the total quantity of books sold by each author

SQL QUERY

```
SELECT B.author, SUM(O.quantity) AS Total_Quantity_Sold
FROM Books AS B
JOIN Orders AS O
ON B.book_id=O.book_id
GROUP BY B.author;
```

QUERY RESULT

	author character varying (50)	total_quantity_sold bigint
1	Jared Cortez	10
2	Tracy Parker	11
3	Taylor Wang	9
4	Cathy Knight	6
5	Bianca Matthews	3
6	Douglas Malone	6

Total rows: 314 Query complete 00:00:00.119

Advanced Query 07) List the cities where customers who spent over $30 are located

SQL QUERY

```
SELECT DISTINCT C.city, O.total_amount
FROM Customers AS C
JOIN Orders AS O
ON C.customer_id=O.customer_id
WHERE O.total_amount>30
ORDER BY O.total_amount;
```

QUERY RESULT

	city character varying (100)	total_amount numeric (10,2)
1	East Tracystad	30.02
2	South Heather	30.02
3	Leeview	30.50
4	Nelsonmouth	31.48
5	Micheleborough	31.50
6	New Beverly	31.68

Total rows: 443 Query complete 00:00:00.1(

Advanced Query 08) Find the customer who spent the most on orders

SQL QUERY

```
SELECT C.customer_id, C.name AS Customer_Name,
SUM(O.total_amount) AS Total_Spent
FROM Customers AS C
JOIN Orders AS O
ON C.customer_id=O.customer_id
GROUP BY C.customer_id, C.name
ORDER BY Total_Spent DESC
LIMIT 1;
```

QUERY RESULT

	customer_id [PK] integer	customer_name character varying (50)	total_spent numeric
1	457	Kim Turner	1398.90

Advanced Query 09) Calculate stock remaining after fulfilling all orders

SQL QUERY

```
SELECT B.book_id, B.title, B.stock, COALESCE(SUM(O.quantity),0) AS Order_quantity,
        B.stock-COALESCE(SUM(O.quantity),0) AS Remaining_Stock
FROM Books AS B
LEFT JOIN Orders AS O
ON B.book_id=O.book_id
GROUP BY B.book_id, B.title
ORDER BY B.book_id;
```

QUERY RESULT

	book_id [PK] integer	title character varying (100)	stock integer	order_quantity bigint	remaining_stock bigint
1	1	Configurable modular throughput	100	3	97
2	2	Persevering reciprocal knowledge user	19	0	19
3	3	Streamlined coherent initiative	27	5	22
4	4	Customizable 24hour product	8	0	8
5	5	Adaptive 5thgeneration encoding	16	8	8
6	6	Advanced encompassing implementation	2	0	2

Total rows: 500 Query complete 00:00:00.147

Key Findings & Learnings

1. **Sales and Revenue Analysis:** Total revenue generated was $75,628.66.

2. **Customer Spending:** Kim Turner (Customer ID: 457) was the highest-spending customer.

3. **Customer Purchasing Behavior:** Identified 139 customers who placed multiple orders and analyzed their spending.

4. **Inventory Management:** Analyzed total inventory and remaining stock after fulfilling orders.

5. **Practical SQL Skills:** Strengthened my knowledge of filtering, sorting, aggregate functions, JOINs, GROUP BY, HAVING, subqueries, and COALESCE.

Thank You

Sanjay Kumar Thakur • SQL Data Analysis Project